Explications du code

TIC315 — Projet 4, jeu d'échecs en ligne. Ce fichier explique simplement chaque partie du code, au fur et à mesure.

Comment le code est rangé

```
code/
├── pom.xml      la liste de ce dont le projet a besoin (Maven)
├── mvnw         lance Maven sans avoir à l'installer
└── metier/      le noyau du jeu : aucune trace de Spring ni du web
    ├── src/main/java/   le code
    └── src/test/java/   les tests JUnit
```

Pour tout compiler et lancer les tests, depuis le dossier `code/` :

```
./mvnw test
```

Partie 1 : les joueurs et le classement

Dossier : `metier/src/main/java/echecs/metier/joueur/`. On commence ici parce que ce package ne dépend d'aucun autre.

Classe	Son rôle
`Joueur`	Un compte : pseudo, e-mail, état, et un classement par famille de cadence
`Classement`	Les points d'un joueur dans une famille. Tout le monde démarre à 1500
`CalculClassement`	Une interface : « donne-moi la variation de points », sans dire comment
`CalculElo`	La formule choisie pour calculer cette variation
`EtatCompte`	`ACTIF` ou `SUSPENDU`
`CategorieCadence`	`BULLET`, `BLITZ` ou `RAPIDE`

Pourquoi ces choix

Pourquoi `CalculClassement` **et** `CalculElo` **sont séparés ?** C'est le patron Stratégie. La partie demandera une variation de points sans connaître la formule. Pour en changer un jour, on ajoutera une classe sans toucher à la partie.

Pourquoi le joueur a un indicateur `occupe` **?** Un joueur ne peut avoir qu'une partie à la fois (RG11). `Joueur` ne connaît pas ses parties, sinon `joueur` et `partie` dépendraient l'un de l'autre. C'est donc la partie qui l'occupe en démarrant et le libère à la fin.

Pourquoi une `EnumMap` **pour les classements ?** Elle associe exactement un `Classement` à chaque `CategorieCadence`. C'est la traduction en Java de l'association qualifiée du diagramme de classes. Les trois classements sont créés avec le joueur, donc jamais absents.

Pourquoi des exceptions ? Une erreur de programmation doit se voir tout de suite plutôt que fausser une partie. Occuper un joueur déjà en partie lève `IllegalStateException` ; créer un joueur sans pseudo lève `IllegalArgumentException`.

La formule Elo, avec un exemple

Un joueur à 1500 bat un joueur à 1480 :

1. Son score attendu est `1 / (1 + 10^((1480 - 1500) / 400))`, soit environ 0,53.
2. Il a marqué 1 (une victoire). Sa variation est `20 × (1 - 0,53)`, soit environ 9,4.
3. On arrondit : il gagne **9 points**, et le perdant en perd **9**.

Battre un joueur beaucoup plus fort rapporte presque 20 points. Battre un joueur beaucoup plus faible ne rapporte presque rien.

Ce que vérifient les tests

Fichier de test	Ce qu'il vérifie
`CalculEloTest`	La formule : victoire, défaite, nulle, joueurs de même niveau, écart important, valeurs refusées
`JoueurTest`	1500 points au départ, classements indépendants (RG6), une partie à la fois (RG11), suspension (RG8), champs obligatoires

Le nom d'un test décrit ce qu'il vérifie, et cite la règle de gestion quand il y en a une : `unCompteSuspenduNePeutPasJouer_RG8`.

Partie 2 : l'échiquier et les règles

Dossier : `metier/src/main/java/echecs/metier/jeu/`. Ce package ne connaît ni joueur ni partie : seulement les règles des échecs.

Classe	Son rôle
`Plateau`	Les 64 cases et les règles qui demandent de voir tout l'échiquier : légalité, échec, mat, pat
`Case`	Une colonne, une rangée, et la pièce qui s'y trouve, s'il y en a une
`Piece`	Ce que toutes les pièces partagent : couleur, « a déjà bougé »
`Roi`, `Dame`, `Tour`, `Fou`, `Cavalier`, `Pion`	Comment chaque pièce se déplace
`Coup`	Un coup joué. Il ne change jamais : l'historique ne se réécrit pas (RG7)
`Direction`	Un pas sur l'échiquier : tant de colonnes, tant de rangées
`Couleur`, `TypePromotion`, `MotifFin`	Les énumérations

Comment on sait qu'un coup est légal

C'est le cœur du jeu, en deux étapes :

1. **La pièce sait où elle peut aller** (`deplacementsPossibles`) : une tour va en ligne droite, jusqu'au premier obstacle. Chaque pièce a sa règle : c'est du polymorphisme, sans aucun `if` sur la sorte de pièce.
2. **Le plateau vérifie que le roi reste à l'abri** (`laisseRoiEnEchec`) : il essaie le coup sur une copie de lui-même et regarde si le roi est attaqué. La vraie partie n'est pas touchée.

Pourquoi ces choix

Pourquoi `casesAttaquees` **est séparée de** `deplacementsPossibles` **?** Pour éviter une boucle sans fin. Pour roquer, il faut savoir si l'adversaire attaque une case. Si cela demandait les coups du roi adverse, qui comprennent son roque, qui demande à son tour les cases attaquées, le calcul ne s'arrêterait jamais. De plus, un pion n'attaque pas là où il avance.

Pourquoi le plateau retient `casePriseEnPassant` **?** La prise en passant n'est possible qu'au coup suivant l'avance de deux cases d'un pion (RG2). Chaque coup efface donc cette case.

Pourquoi `Optional` **?** Un retour `[0..1]` du diagramme devient `Optional<Piece>` : le code qui l'appelle est obligé de traiter le cas où il n'y a rien, au lieu d'oublier un `null`.

Pourquoi la partie n'est-elle pas ici ? Le trait, l'historique, la pendule, la triple répétition et les 50 coups regardent la partie : ils viendront dans `metier.partie`.

Limites connues

- La notation d'un coup (`Cf3`, `exd5`) ne précise pas laquelle de deux pièces a joué, et n'indique pas l'échec.
- `peutMater` et le matériel insuffisant traitent les cas courants : roi seul, roi et une pièce légère, fous sur des cases de même couleur. Les fins de partie plus rares sont hors périmètre.
- Les 13 tests perft prennent environ 35 secondes : essayer chaque coup sur une copie du plateau est simple, mais pas rapide. C'est sans effet pour jouer une partie.

Ce que vérifient les tests

Fichier de test	Ce qu'il vérifie
`PlateauTest`	Coup illégal, pièce clouée, roque, prise en passant, promotion, mat, pat, matériel insuffisant, position FEN
`CoupTest`	La notation, et ce qui remet à zéro la règle des 50 coups
`PerftTest`	Le nombre total de coups possibles, comparé à des résultats publiés

Perft est la méthode de référence : on compte toutes les suites de coups possibles sur quelques demi-coups, et on compare à des résultats connus de tous les programmeurs d'échecs. Par exemple, il y a exactement 8 902 suites de 3 demi-coups depuis la position de départ. Une seule erreur sur le roque ou la prise en passant change le total : nous l'avons vérifié en cassant volontairement ces deux règles.

Partie 3 : la partie

Dossier : `metier/src/main/java/echecs/metier/partie/`. C'est le chef d'orchestre : il utilise l'échiquier et les joueurs, et applique le déroulement d'une partie. Le noyau métier est ainsi complet.

Classe	Son rôle
`Partie`	Ce que toute partie partage : le trait, les coups, la fin, les observateurs
`PartieEntreJoueurs`	Ajoute le créateur, l'invité, la pendule, la nulle et le classement
`PartieContreOrdinateur`	Sans pendule ni classement : elle commence tout de suite
`Participation`	Un joueur dans une partie : sa couleur et sa pendule

Classe	Son rôle
`Cadence`	Le temps de base, l'incrément, la famille
`EtatDePartie` et ses 4 états	Ce qui est permis à chaque moment (patron État)
`ObservateurPartie`	Ce que doit savoir faire une vue pour être prévenue (patron Observateur)
`CoupRefuseException`, `TransitionInterditeException`	Les deux façons de dire non

Le patron État, en clair

Une partie ne réagit pas pareil selon son état : on ne peut pas jouer dans une partie terminée. Sans patron, chaque méthode commencerait par des `if (etat == ...)`. Ici, `Partie` transmet la demande à son état courant :

```
public void abandonner(Joueur joueur) {
    etatCourant.abandonner(this, joueur);
    notifierObservateurs();
}
```

Par défaut, `EtatDePartie` refuse tout. `EtatEnCours` redéfinit `abandonner` et termine la partie. `EtatTerminee` ne redéfinit rien : il refuse donc tout, sans une seule ligne de code en plus.

Pourquoi deux exceptions ?

- `TransitionInterditeException` : ce n'est pas le moment, ou pas à toi. Exemple : abandonner une partie terminée.
- `CoupRefuseException` : la partie accepte des coups, mais pas celui-là. Exemple : le coup est illégal, ou ce n'est pas ton tour.

Pourquoi ces choix

Pourquoi une `Clock` ? La pendule a besoin de l'heure. Avec une horloge passée en paramètre, un test avance le temps de dix minutes d'un coup, sans attendre : c'est l'injection de dépendances.

Pourquoi `apresCoup()` et `mettreAJourClassements()` sont vides dans `Partie` ? Seule la partie entre joueurs a une pendule et un classement : elle seule les redéfinit. C'est du polymorphisme, sans aucun test sur la sorte de partie.

Pourquoi le coup de l'ordinateur passe par le même chemin ? Il subit les mêmes vérifications : si Stockfish renvoyait un coup illégal, il serait refusé (UC7, exception E2).

Pourquoi `verifierTemps` avant chaque coup ? Un coup joué après la chute du drapeau ne doit pas compter (RG5). La partie le vérifie elle-même, sans dépendre du bon vouloir de l'appelant.

Limite connue

`partie.plateau()` donne accès au plateau, y compris à `deplacer`. Seule la partie doit déplacer les pièces : c'est une règle à respecter, que le compilateur n'impose pas.

Ce que vérifient les tests

Fichier de test	Ce qu'il vérifie
`PartieEntreJoueursTest`	Rejoindre, tour de jeu, pendule, mat, abandon, nulles, répétition, 50 coups, classement, observateurs

Fichier de test	Ce qu'il vérifie
`EtatsPartieTest`	Ce que chaque état accepte et refuse, l'annulation
`PartieContreOrdinateurTest`	Sans pendule, non classée, coup du moteur vérifié, moteur muet
`ParticipationTest`	Le calcul du temps restant

Le test des 50 coups joue une vraie partie : 4 coups de pion, puis des tours qui font des allers-retours en parcours de longueurs différentes, pour qu'aucune position ne revienne trois fois avant le 100e demi-coup.

Partie 4 : la couche de contrôle

Dossier : `controle/src/main/java/echecs/controle/`. C'est le serveur du restaurant : il reçoit les demandes et les transmet au métier, sans jouer aux échecs lui-même. Il ne contient aucune trace de Spring, de la base de données ni de Stockfish : il ne connaît que des interfaces.

Classe	Son rôle
`ControleurPartie`	Un par joueur connecté : reçoit ses actions de jeu et les transmet à sa partie
`ControleurCompte`	Créer un compte, se connecter, suspendre, réactiver
`FilesDeParties`	La file d'actions de chaque partie
`ArchivagePartie`	Un observateur qui enregistre la partie quand elle se termine
`IPartie`, `ICompte`	Ce que le site a le droit de demander
`MoteurEchecs`, `ServiceAuthentification`, `DepotParties`, `DepotJoueurs`	Ce dont le contrôle a besoin, sans savoir qui le fournit

La file d'actions, en clair

Le serveur traite chaque joueur en parallèle. Sans précaution, deux joueurs pourraient modifier la même partie au même instant, et un coup pourrait s'enregistrer dans une partie déjà terminée.

Chaque partie a donc sa file, comme un arbitre qui prend les demandes une par une :

```
files.executer(partie, () -> partie.abandonner(joueurConnecte));
```

L'action attend son tour, s'exécute, puis l'appelant reçoit le résultat ou l'exception. Les autres parties ne sont pas ralenties, car chacune a sa propre file.

Stockfish réfléchit hors de la file

Si Stockfish réfléchissait dans la file, la partie serait bloquée pendant son calcul : le joueur ne pourrait même pas abandonner. Le travail se fait donc en trois temps, dans `faireJouerOrdinateur` :

1. lire la position, dans la file ;
2. demander le coup à Stockfish, **hors de la file** ;
3. jouer ce coup, dans la file.

Si le joueur abandonne entre-temps, l'étape 3 est refusée, et le coup tardif est simplement ignoré.

Pourquoi ces choix

Pourquoi `jouerCoup` reçoit du texte (`"e2"`) ? Le navigateur n'envoie que du texte. Le nom est vérifié ici, à la frontière : `z9` est refusé avant d'atteindre le métier.

Pourquoi demander deux fois au moteur ? La fiche UC7 prévoit de renouveler la demande une fois. Après un second échec, la partie est annulée, sans résultat (RG15).

Pourquoi un joueur est-il identifié par son pseudo ? Le pseudo est unique (RG12) : un joueur rechargé depuis la base reste le même joueur pour la partie.

Pourquoi le dépôt doit-il renvoyer le même objet ? L'indicateur « occupé » n'existe qu'en mémoire. Deux objets pour un même joueur croiraient chacun qu'il est libre, et RG11 ne serait plus respectée. C'est écrit dans `DepotJoueurs`, et ce sera à respecter dans la persistance.

Limites connues, pour la suite

- **Qui a le droit de suspendre un compte ?** `ControleurCompte::suspendre` ne vérifie pas que l'appelant est administrateur, et le modèle n'a pas encore cette notion. Le site devra l'imposer : sans cela, n'importe quel joueur pourrait suspendre les autres.
- **Une partie en cours est perdue si le serveur redémarre**, comme décidé : elle n'existe qu'en mémoire.
- **Les pendules** ne tombent toutes seules que si le site appelle `verifierToutesLesPendules` régulièrement.
- **L'unicité des comptes** (RG12) devra aussi être garantie par la base de données : deux requêtes simultanées peuvent passer la vérification du contrôleur.

Ce que vérifient les tests

Fichier de test	Ce qu'il vérifie
`FilesDePartiesTest`	32 fils jouent le même coup en même temps : un seul passe. Une partie lente ne bloque pas les autres
`ControleurPartieTest`	Le parcours complet entre deux joueurs, le coup de l'ordinateur, le moteur muet, la promotion, l'abandon pendant que le moteur réfléchit
`ControleurCompteTest`	Comptes, connexion, suspension, réactivation

Ces tests utilisent de faux dépôts et un faux moteur : c'est l'avantage de l'injection de dépendances. Stockfish et la base réelle ne sont donc pas encore testés.